

Raft: Elasticity Challenge

- Configuration change to replace servers when they fail or to change the degree of replication.
- Simple solution: Take the cluster off-line, update configuration files, and then restart the cluster.
 - Disadvantage: Data becomes unavailable during the changeover. Manual steps may risk operator error.
- On-line configuration change:
 - Requirement: No instance of time with two leaders for the same term.
 - Switch from the old configuration to the new configuration is unsafe. Figure 10 shows this in the form of 2 different leaders for the same term.

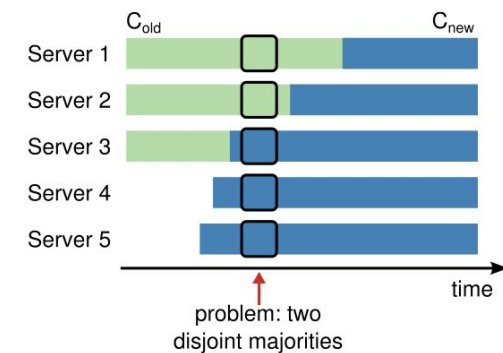


Figure 10: Switching directly from one configuration to another is unsafe because different servers will switch at different times. In this example, the cluster grows from three servers to five. Unfortunately, there is a point in time where two different leaders can be elected for the same term, one with a majority of the old configuration (C_{old}) and another with a majority of the new configuration (C_{new}).

Raft: Elasticity Solution

- A 2-phase approach:
 - Phase 1: Switch to a transitional configuration termed joint consensus.
 - Phase 2: Commit the joint consensus and transition to the new configuration.
- Joint consensus allows individual servers to transition from old to new at different times without compromising safety and while serving requests. It combines both the old and new configurations:
 - Log entries are replicated to all servers in both configurations.
 - Any server from either configuration may serve as leader.
 - Agreement for election and entry commitment requires separate majorities from **both** the old and new configurations.

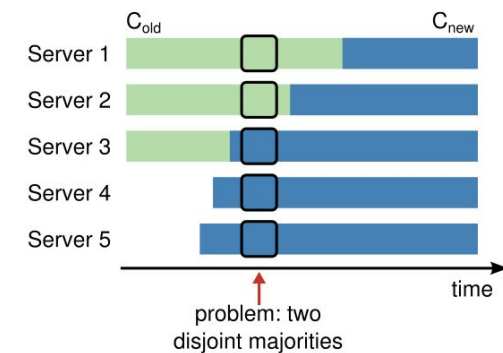


Figure 10: Switching directly from one configuration to another is unsafe because different servers will switch at different times. In this example, the cluster grows from three servers to five. Unfortunately, there is a point in time where two different leaders can be elected for the same term, one with a majority of the old configuration (C_{old}) and another with a majority of the new configuration (C_{new}).

Raft: Cluster Configurations

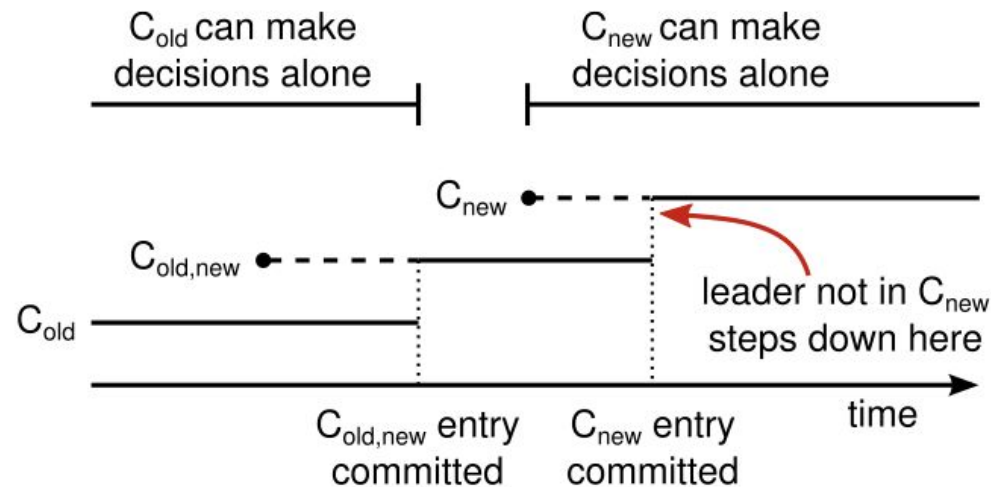
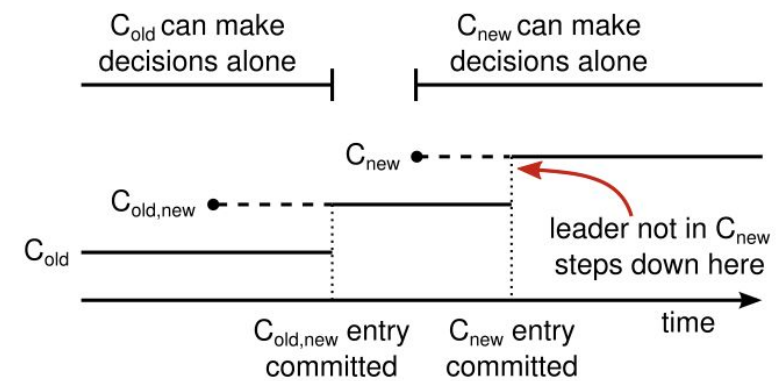


Figure 11: Timeline for a configuration change. Dashed lines show configuration entries that have been created but not committed, and solid lines show the latest committed configuration entry. The leader first creates the $C_{old,new}$ configuration entry in its log and commits it to $C_{old,new}$ (a majority of C_{old} and a majority of C_{new}). Then it creates the C_{new} entry and commits it to a majority of C_{new} . There is no point in time in which C_{old} and C_{new} can both make decisions independently.

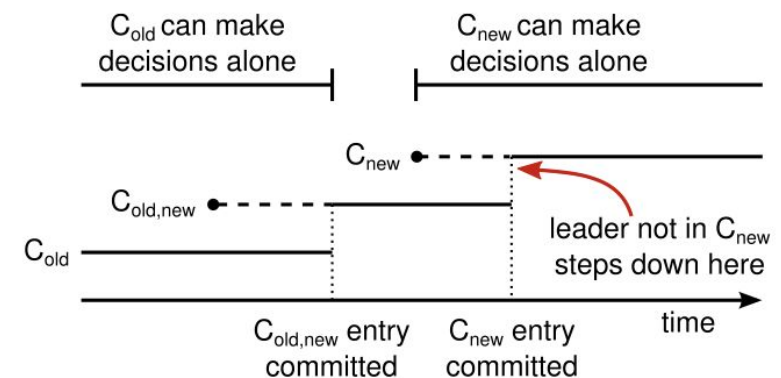
Raft: C_{old} to C_{new}

trates the configuration change process. When the leader receives a request to change the configuration from C_{old} to C_{new} , it stores the configuration for joint consensus ($C_{old,new}$ in the figure) as a log entry and replicates that entry using the mechanisms described previously. Once a given server adds the new configuration entry to its log, it uses that configuration for all future decisions (a server always uses the latest configuration in its log, regardless of whether the entry is committed). This means that the leader will use the rules of $C_{old,new}$ to determine when the log entry for $C_{old,new}$ is committed. If the leader crashes, a new leader may be chosen under either C_{old} or $C_{old,new}$, depending on whether the winning candidate has received $C_{old,new}$. In any case, C_{new} cannot make unilateral decisions during this period.



Raft: $C_{old,new}$ to C_{new}

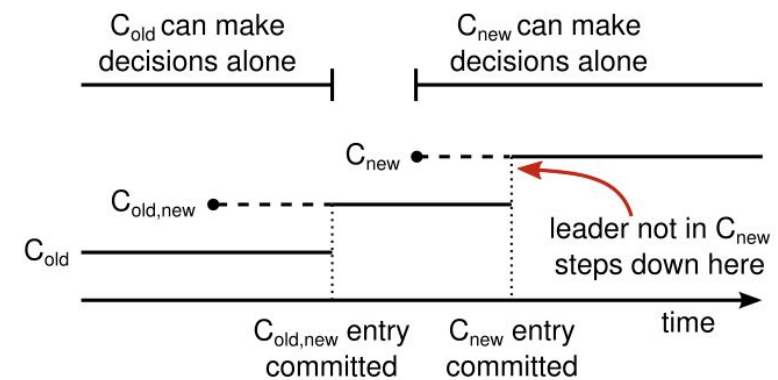
Once $C_{old,new}$ has been committed, neither C_{old} nor C_{new} can make decisions without approval of the other, and the Leader Completeness Property ensures that only servers with the $C_{old,new}$ log entry can be elected as leader. It is now safe for the leader to create a log entry describing C_{new} and replicate it to the cluster. Again, this configuration will take effect on each server as soon as it is seen. When the new configuration has been committed under the rules of C_{new} , the old configuration is irrelevant and servers not in the new configuration can be shut down. As



There is no time when C_{old} and C_{new} are both able to make decisions. This guarantees safety.

Raft: Non-Voting Members

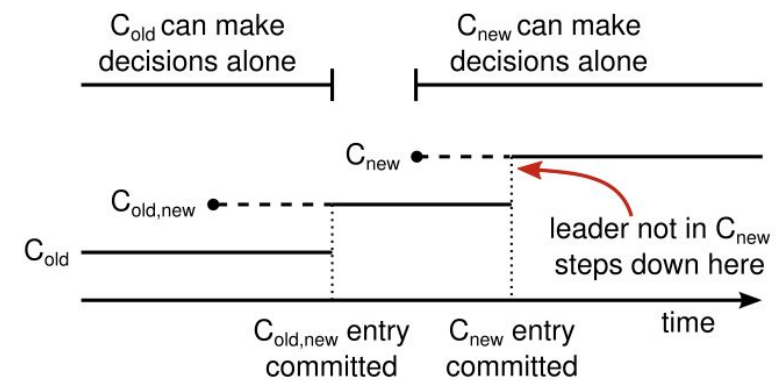
Raft introduces an additional phase before the configuration change, in which the new servers join the cluster as non-voting members (the leader replicates log entries to them, but they are not considered for majorities). Once the new servers have caught up with the rest of the cluster, the reconfiguration can proceed as described above.



Why? New servers may not initially store log entries. Adding such servers to a cluster may require a while for them to catch up. During this time, when the new servers are the majority, it may not be possible to commit new log entries.

Raft: What if we want to remove the leader?

The second issue is that the cluster leader may not be part of the new configuration. In this case, the leader steps down (returns to follower state) once it has committed the C_{new} log entry. This means that there will be a period of time (while it is committing C_{new}) when the leader is managing a cluster that does not include itself; it replicates log entries but does not count itself in majorities. The leader transition occurs when C_{new} is committed because this is the first point when the new configuration can operate independently (it will always be possible to choose a leader from C_{new}). Before this point, it may be the case that only a server from C_{old} can be elected leader.



Raft: Removed Servers May Disrupt the Cluster

The third issue is that removed servers (those not in C_{new}) can disrupt the cluster. These servers will not receive heartbeats, so they will time out and start new elections. They will then send RequestVote RPCs with new term numbers, and this will cause the current leader to revert to follower state. A new leader will eventually be elected, but the removed servers will time out again and the process will repeat, resulting in poor availability.

To prevent this problem, servers disregard RequestVote RPCs when they believe a current leader exists. Specifically, if a server receives a RequestVote RPC within the minimum election timeout of hearing from a current leader, it does not update its term or grant its vote.

This does not affect normal elections, where each server waits at least a minimum election timeout before starting an election. However, it helps avoid disruptions from removed servers: if a leader is able to get heartbeats to its cluster, then it will not be deposed by larger term numbers.

